

EXPERIMENT NO. 3

ABISHEK DEVARAJ (192512126)

DEVICE ONBOARDING AND DATA FLOW IN AWS IoT CORE – GOOGLE COLAB SIMULATION

AIM

To simulate IoT device onboarding and cloud data flow similar to AWS IoT Core using Python in Google Colab for a Smart Parking System.

OBJECTIVES

1. To simulate IoT device registration.
2. To authenticate IoT devices.
3. To generate smart parking occupancy data.
4. To simulate transmission of telemetry data to the cloud.
5. To perform stream analytics on parking data.
6. To calculate parking occupancy.
7. To detect parking congestion.
8. To visualize parking occupancy.
9. To store the collected parking data in CSV format.

SOFTWARE / APPARATUS REQUIRED

- Google Colab
- Python 3
- Pandas
- NumPy
- Matplotlib

THEORY

AWS IoT Core is a managed cloud service that enables IoT devices to securely connect with cloud applications and exchange data.

A typical AWS IoT Core data flow is:

IoT Device → Authentication → AWS IoT Core → Cloud Storage → Analytics → Dashboard

Before sending data, an IoT device is normally registered with the cloud platform. Authentication mechanisms such as certificates and security policies are used to verify the device.

In this experiment, the AWS IoT Core environment is simulated using Python in Google Colab.

The simulation consists of the following stages:

1. Device Registration

A unique device ID is created for the smart parking device.

2. Device Authentication

The device is checked against the registered device information. If the device is valid, authentication is successful.

3. Telemetry Generation

The simulated parking device generates occupancy information for individual parking slots.

4. Cloud Upload

The generated parking data is treated as telemetry transmitted to a simulated cloud environment.

5. Stream Analytics

The number of occupied and vacant slots is calculated. The occupancy percentage is also determined.

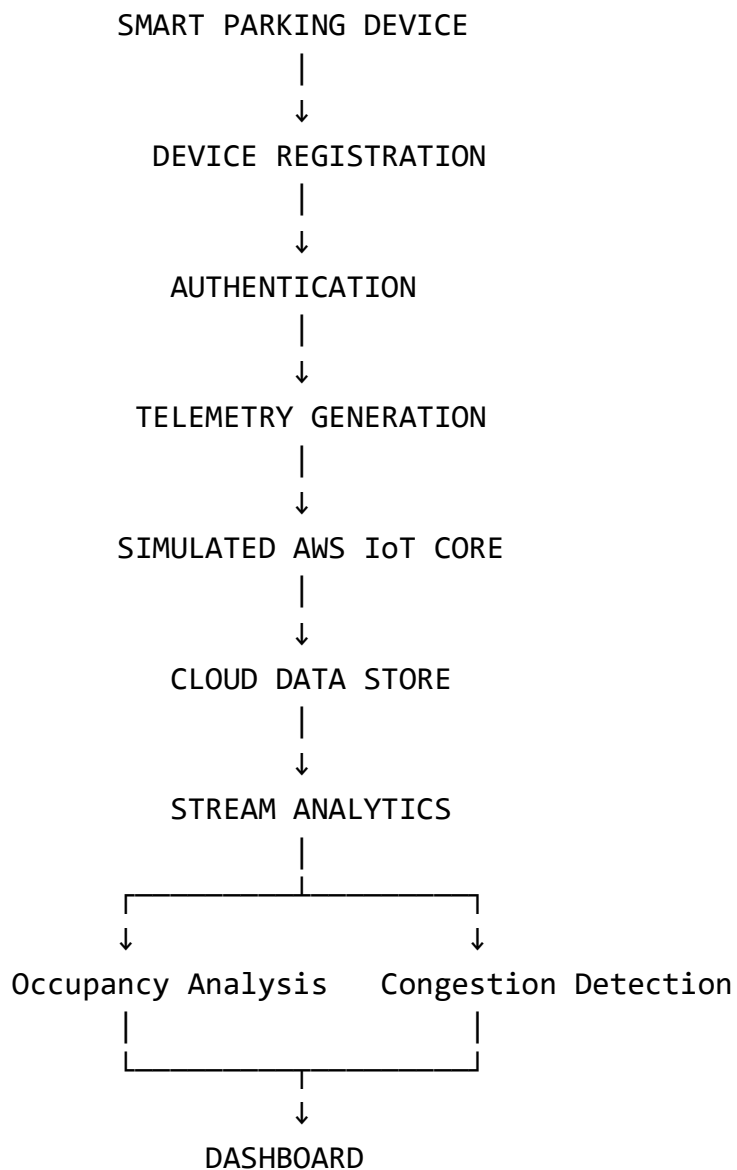
6. Congestion Detection

If the parking occupancy exceeds the defined threshold, the system identifies the parking area as congested.

7. Data Storage

The collected parking telemetry is stored in a CSV file for future analysis.

SYSTEM FLOW



ALGORITHM

1. Start the program.
2. Import the required Python libraries.
3. Create a unique Smart Parking IoT device ID.
4. Register the IoT device.
5. Authenticate the registered device.
6. Define the total number of parking slots.
7. Generate the occupancy status of each parking slot.
8. Generate occupied and vacant slot information.
9. Create the parking telemetry dataset.
10. Simulate transmission of the telemetry data to the cloud.
11. Calculate the total number of occupied slots.
12. Calculate the total number of vacant slots.
13. Calculate the parking occupancy percentage.
14. Compare the occupancy percentage with the congestion threshold.
15. Display the parking status.
16. Store the telemetry data in a CSV file.
17. Plot the parking occupancy graph.
18. Stop the program.

GOOGLE COLAB PROGRAM

```
import random

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
# =====
```

```

# DEVICE ONBOARDING

# =====

device_id = "SMART_PARKING_DEVICE_03"


print("=" * 65)
print("AWS IoT CORE - SMART PARKING SIMULATION")
print("=" * 65)


print("\nDEVICE ONBOARDING")
print("-" * 65)


print("Device ID      :", device_id)
print("Device Type     : Smart Parking Sensor")
print("Location        : Parking Area A")
print("Registration     : SUCCESS")


# =====

# DEVICE AUTHENTICATION

# =====


print("\nDEVICE AUTHENTICATION")
print("-" * 65)


print("Device ID      :", device_id)
print("Authentication : SUCCESS")

```

```
print("Cloud Access    : GRANTED")

# =====
# PARKING CONFIGURATION
# =====

total_slots = 80

occupancy_probability = 0.65

congestion_threshold = 75

# =====
# GENERATE PARKING DATA
# =====

slot_numbers = []
status = []
occupancy_value = []

for slot in range(1, total_slots + 1):

    if random.random() < occupancy_probability:

        slot_status = "Occupied"
```

```

        value = 1

    else:

        slot_status = "Vacant"
        value = 0

    slot_numbers.append(slot)
    status.append(slot_status)
    occupancy_value.append(value)

# =====
# CREATE DATAFRAME
# =====

df = pd.DataFrame({

    "Device_ID": [device_id] * total_slots,

    "Parking_Slot": slot_numbers,

    "Status": status,

    "Occupancy_Value": occupancy_value

})

```

```

# =====
# CLOUD DATA FLOW
# =====

print("\nCLOUD DATA FLOW")
print("-" * 65)

print("Device          :", device_id)
print("Telemetry Generation : SUCCESS")
print("Cloud Upload        : SUCCESS")
print("Total Records       :", len(df))

# =====
# STREAM ANALYTICS
# =====

occupied = (df["Status"] == "Occupied").sum()

vacant = (df["Status"] == "Vacant").sum()

occupancy_percentage = (
    occupied / total_slots
) * 100

```



```

vacancy_percentage = (
    vacant / total_slots
) * 100

# =====
# CONGESTION DETECTION
# =====

if occupancy_percentage >= congestion_threshold:

    parking_status = "HIGH CONGESTION"

else:

    parking_status = "NORMAL"

# =====
# PARKING SUMMARY
# =====

print("\n" + "=" * 65)
print("SMART PARKING ANALYTICS")
print("=" * 65)

print("\nTotal Parking Slots  :", total_slots)

```

```
print("Occupied Slots      :", occupied)
```

```
print("Vacant Slots       :", vacant)
```

```
print(  
    "Occupancy Percentage :",  
    round(occupancy_percentage, 2),  
    "%"  
)
```

```
print(  
    "Vacancy Percentage   :",  
    round(vacancy_percentage, 2),  
    "%"  
)
```

```
print(  
    "Congestion Threshold :",  
    congestion_threshold,  
    "%"  
)
```

```
print(  
    "Parking Status      :",  
    parking_status  
)
```

```

# =====

# DISPLAY PARKING DATA

# =====

print("\nPARKING SLOT TELEMETRY")
print("-" * 65)

print(df.to_string(index=False))

# =====

# VACANT AND OCCUPIED SLOTS

# =====

vacant_slots = df[
    df["Status"] == "Vacant"
][["Parking_Slot"].tolist()

occupied_slots = df[
    df["Status"] == "Occupied"
][["Parking_Slot"].tolist()

print("\nVACANT SLOT NUMBERS")
print("-" * 65)

```

```
print(vacant_slots)
```

```
print("\nOCCUPIED SLOT NUMBERS")
```

```
print("-" * 65)
```

```
print(occupied_slots)
```

```
# =====
```

```
# SAVE DATA
```

```
# =====
```

```
file_name = "smart_parking_cloud_data_03.csv"
```

```
df.to_csv(  
    file_name,  
    index=False  
)
```

```
print("\nDATA STORAGE")
```

```
print("-" * 65)
```

```
print("CSV File Created :", file_name)
```

```
# =====  
# PARKING OCCUPANCY GRAPH  
# =====  
  
plt.figure(figsize=(14, 5))  
  
bar_colors = [  
    "red" if x == "Occupied"  
    else "green"  
    for x in status  
]  
  
plt.bar(  
    slot_numbers,  
    [1] * total_slots,  
    color=bar_colors  
)  
  
plt.xlabel("Parking Slot Number")  
  
plt.ylabel("Occupancy")  
  
plt.title(  
    "Smart Parking Occupancy - AWS IoT Core Simulation"  
)  
  
plt.yticks(
```

```

        [0, 1],
        ["Vacant", "Occupied"]
    )

plt.grid(
    axis="y",
    alpha=0.3
)

plt.tight_layout()

plt.show()

# =====
# SUMMARY GRAPH
# =====

plt.figure(figsize=(7, 5))

plt.bar(
    ["Occupied", "Vacant"],
    [occupied, vacant]
)

plt.xlabel("Parking Status")

```

```
plt.ylabel("Number of Slots")
```

```
plt.title(  
    "Smart Parking Occupancy Summary"  
)
```

```
plt.grid(  
    axis="y",  
    alpha=0.3  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# FINAL RESULT  
# =====
```

```
print("\n" + "=" * 65)  
print("EXPERIMENT COMPLETED SUCCESSFULLY")  
print("=" * 65)
```

```
print("Device Onboarding          : SUCCESS")  
print("Device Authentication       : SUCCESS")  
print("Telemetry Generation         : SUCCESS")
```

```
print("Cloud Upload Simulation    : SUCCESS")
print("Stream Analytics           : SUCCESS")
print("Congestion Detection        : SUCCESS")
print("Data Storage                 : SUCCESS")
print("Visualization                : SUCCESS")
```

SAMPLE OUTPUT

Important: The values below are only an example. When the program is executed in Google Colab, the parking-slot values will be randomly generated, so the actual output will be different.

```
=====
=
AWS IoT CORE - SMART PARKING SIMULATION
=====
=

DEVICE ONBOARDING
-----
-
Device ID       : SMART_PARKING_DEVICE_03
Device Type     : Smart Parking Sensor
Location        : Parking Area A
Registration    : Registered

DEVICE AUTHENTICATION
-----
-
Device ID       : SMART_PARKING_DEVICE_03
Authentication  : SUCCESS
Cloud Access    : GRANTED

CLOUD DATA FLOW
-----
-
Device          : SMART_PARKING_DEVICE_03
Telemetry Status : Data Generated
Cloud Connection : Simulated
```


Cloud Upload : SUCCESS
Total Records : 80

=====

=

SMART PARKING ANALYTICS

=====

=

Total Parking Slots : 80
Occupied Slots : 53
Vacant Slots : 27
Occupancy Percentage : 66.25 %
Vacancy Percentage : 33.75 %
Congestion Threshold : 75 %
Parking Status : NORMAL

The slot telemetry will appear in the form:

Device_ID	Parking_Slot	Status	Occupancy_Value
SMART_PARKING_DEVICE_03	1	Occupied	1
SMART_PARKING_DEVICE_03	2	Vacant	0
SMART_PARKING_DEVICE_03	3	Occupied	1
SMART_PARKING_DEVICE_03	4	Occupied	1
SMART_PARKING_DEVICE_03	5	Vacant	0
...			

The exact values will be different every time because the parking occupancy is randomly generated.

EXPECTED GRAPH OUTPUT

The first graph represents the occupancy status of every parking slot.

- **Red bar** → Occupied
- **Green bar** → Vacant

The second graph shows the total number of occupied and vacant parking slots.

OBSERVATION

Parameter	Observed Value
Device ID	SMART_PARKING_DEVICE_03
Total Parking Slots	80
Occupied Slots	Randomly generated
Vacant Slots	Randomly generated
Occupancy Percentage	Calculated dynamically
Congestion Threshold	75%
Parking Status	Normal / High Congestion
Cloud Upload	Successful
CSV Storage	Successful
	OUT

RESULT

The Smart Parking IoT system was successfully simulated using Python in Google Colab. The IoT device was registered and authenticated before transmitting simulated parking telemetry data. The parking occupancy information was processed as cloud data similar to the data flow in AWS IoT Core.

The system successfully calculated the number of occupied and vacant parking slots, determined the occupancy percentage, detected parking congestion, visualized the parking status, and stored the collected telemetry data in CSV format.

Thus, device onboarding and IoT cloud data flow similar to AWS IoT Core were successfully simulated for a Smart Parking System using Google Colab.